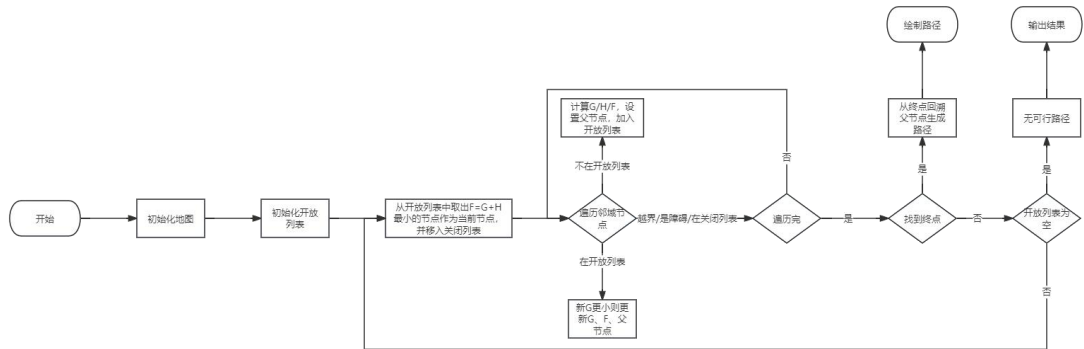


实验四实验报告

流程框图：



伪代码：

// 全局定义

MAP_WIDTH = 80, MAP_HEIGHT = 80 // 地图尺寸

TILE_SIZE = 32 // 网格像素大小

// 移动代价：直线 10，斜线 14

COST_STRAIGHT = 10, COST_DIAGONAL = 14

// 节点结构

STRUCT Node

x, y: 坐标

g: 从起点到当前节点的代价

h: 启发函数

parent: 父节点指针

next: 链表下一个节点

END STRUCT

// 启发函数 H

FUNCTION CalculateH(x, y, endX, endY)

dx = 绝对值(x - endX)

dy = 绝对值(y - endY)

IF dx > dy THEN

RETURN 14*dy + 10*(dx-dy)

ELSE

RETURN 14*dx + 10*(dy-dx)

END FUNCTION

// 生成 8 个邻域节点

FUNCTION Expand8Neighbors(x, y)

RETURN [

```

        (x-1,y-1), (x,y-1), (x+1,y-1),
        (x-1,y), (x+1,y),
        (x-1,y+1), (x,y+1), (x+1,y+1)
    ]
END FUNCTION

// A* 寻路
FUNCTION AStarPathFind(startX, startY, endX, endY, map)
    // 1. 初始化列表
    openList = 空链表
    closeList = 空链表
    startNode = new Node()
    startNode.x = startX, startNode.y = startY
    startNode.g = 0, startNode.h = CalculateH(startX, startY, endX, endY)
    startNode.parent = NULL
    add startNode to openList

    // 2. 主循环
    WHILE openList 不为空
        // 取 F 最小的节点
        currentNode = 从 openList 移除 F 最小的节点
        add currentNode to closeList

        // 到达终点
        IF currentNode.x == endX AND currentNode.y == endY THEN
            RETURN 生成路径(currentNode)

        // 遍历 8 邻域
        neighbors = Expand8Neighbors(currentNode.x, currentNode.y)
        FOR EACH (nx, ny) IN neighbors DO
            // 越界/障碍判断
            IF nx<0 OR ny<0 OR nx>=MAP_WIDTH OR ny>=MAP_HEIGHT OR map[ny][nx]
== 障碍 THEN
                CONTINUE
            END IF

            // 在关闭列表，跳过
            IF (nx, ny) IN closeList THEN
                CONTINUE
            END IF

            // 计算新 G
            IF 是斜线移动 THEN
                newG = currentNode.g + COST_DIAGONAL

```

```

ELSE
    newG = currentNode.g + COST_STRAIGHT
END IF

// 邻域节点不在开放列表
IF (nx, ny) NOT IN openList THEN
    newNode = new Node()
    newNode.x = nx, newNode.y = ny
    newNode.g = newG
    newNode.h = CalculateH(nx, ny, endX, endY)
    newNode.parent = currentNode
    add newNode to openList
ELSE
    // 已在开放列表，更新更优路径
    existingNode = 查找 openList 中的(nx, ny)
    IF newG < existingNode.g THEN
        existingNode.g = newG
        existingNode.parent = currentNode
    END IF
END IF
END FOR
END WHILE

// 无路径
RETURN NULL
END FUNCTION

// 从终点回溯生成路径
FUNCTION MakePath(endNode)
    pathList = 空列表
    tempNode = endNode
    WHILE tempNode != NULL
        add tempNode.x, tempNode.y to pathList
        tempNode = tempNode.parent
    END WHILE
    反转 pathList, 得到起点
    RETURN pathList
END FUNCTION

```